



Welcome to the final part of our adventure game series! In this tutorial, we will focus on creating two engaging 3D levels for our game. The first level will be imported from Blender, while the second will be created directly within Godot using grid maps. Additionally, we will enhance our game's visual appeal by creating shaders for water, grass, and lava. This tutorial builds upon the first two parts of the series, where we developed a base game and an inventory system. By combining all three parts, you will have a fully fledged, complex game.

Course Overview

Let's dive into more detail about what we will accomplish in this tutorial:

- Create two levels: an overworld imported from Blender and a modular castle level using Godot's grid maps.
- Enhance the visual appeal of the overworld by adding water and grass via shaders.
- Design a castle level with lava features.
- Implement a system to transition between the two levels.

Game Scenes

Our game will consist of two main scenes:

1. **Overworld:** This scene will serve as the primary environment where players start their adventure.
2. **Castle:** Players will transition to this scene when they reach the castle door in the overworld. They can return to the overworld by interacting with a specific door within the castle.

This approach adds variety and depth to the gameplay experience.

Learning Objectives

By the end of this course, you will have learned the following key concepts:

- 3D level creation in Godot, including importing Blender files and using grid maps.
- Using shaders to add visual details such as water, lava, and grass.
- Loading and transitioning between scenes within the game.

Prerequisites

Before starting this tutorial, you should have a basic understanding of Godot and shaders. While most of the tutorial is straightforward, we will create two spatial shaders—one for water and lava, and another for grass. Familiarity with basic shader concepts like albedo and vertex will be beneficial.

Additionally, this is the third course in our series. In the previous parts, we created a basic game and an inventory system. We will continue to build upon these foundations in this tutorial.

Upcoming Content

In the next part, we will review what we have accomplished so far and prepare to integrate our new levels. While the levels are fairly independent of the previous projects, having a basic understanding of the existing game structure will be helpful.

Get ready to elevate your game development skills and create immersive 3D levels with Godot!

Course Updated to Godot 4.5

We've updated the project files to Godot version 4.5 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

**Godot Engine – .NET****4.5**

x86_64 · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?

Download Godot 4 for macOS

 **Godot Engine** **4.5**

arm64 (Apple Silicon) · x86_64 (Intel) · 15 September 2025

 **Godot Engine – .NET** **4.5**

arm64 (Apple Silicon) · x86_64 (Intel) · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?

If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:

Supported platforms

 **Android**  **Linux**  **macOS**  **Windows**  **Web Editor**



This course builds upon a project from Unit 1 and Unit 2 so you can focus on the level creation. If you skipped these previous units, you can download a project starter file from the Course Files tab located at the bottom of any lesson page or the course's home page!

Need a visual guide? Try out our helpful article on the topic: [How can I download the source code, or asset files, for my course?](#)



Welcome to the third part of our three-part series on creating a basic 3D game with a menu and levels using Godot. In this tutorial, we will focus on creating the levels for our game. Even if you are new to this series, the logic for the levels is fairly contained, so you can follow along easily. However, having a basic understanding of the project from the previous parts will be beneficial.

Prerequisites

Before we dive into creating levels, let's briefly recap what we have accomplished so far:

- Created a basic 3D game with a character class that serves as the parent class for the player and enemies.
- Implemented functionalities like equipping different weapons, shields, and style items.
- Developed an inventory system for equipping different items and previewing the character.
- Added randomness to enemy appearances and weapons.

Project Overview

The current project structure includes several key components:

- **Game Scene:** Contains the environment, lighting, terrain, characters (player and skeleton), a HUD, and the inventory system.
- **Character Class:** Includes the character body, collision shape, animation tree, hit timer, and hit sound. This class has functions for movement, jumping, animations, attacking, defending, and managing equipment.
- **Player Class:** Inherits from the character class and includes additional features like a camera controller and specific animations.
- **Inventory System:** Displays player items and allows equipping different weapons, shields, and style items.
- **Weapon Scenes:** Each weapon or item has its own scene, including a model, Raycast 3D for detecting hits, and sound effects.

Understanding the Character Class

The character class is the backbone of our game. Here's a breakdown of its components:

- **Root Node:** CharacterBody
- **Collision Shape:** Defines the character's collision area.
- **Animation Tree:** Manages different animations like idle, running, jumping, defending, and attacking.
- **Hit Timer:** Ensures the character can't be hit too quickly in succession.
- **Hit Sound:** Plays a sound when the character is hit.

Player Class Specifics

The player class extends the character class and includes additional features:

- **Skin:** The player's appearance, which can be customized.
- **Camera Controller:** Manages the camera's behavior.
- **Animation Tree:** Includes additional animations specific to the player.

Inventory System

The inventory system allows the player to equip different items. Here's how it works:

- The player's equipment is stored in arrays for weapons, shields, and style items.



- The currently equipped item is determined by an index value.
- The inventory scene displays the player's items and allows for equipping and previewing them.

Weapon Scenes

Each weapon or item has its own scene with the following components:

- **Node3D:** The root node of the scene.
- **Model:** The visual representation of the item.
- **Raycast 3D:** Detects when the weapon hits something.
- **Sound:** Plays a sound when the weapon is used.

Global Dictionary

The global.gd file contains dictionaries for weapons, shields, and style items. Each entry includes data such as:

- Weapon type
- Damage
- Thumbnail for the inventory
- Scene to display in the game
- Animation for attacking
- Weapon range
- Audio file

New Additions

If you are following from the second part, there are some minor changes to note:

- Two new folders in the graphics folder: **objects** and **dungeon_grid_map**.
- The **objects** folder contains basic 3D objects like a barrel.
- The **dungeon_grid_map** folder contains dungeon tiles for creating levels.

Conclusion

With this overview, you should have a good understanding of the project's structure and the key components involved. In the following sections, we will dive into creating the levels for our game. Stay tuned for more detailed instructions on designing and implementing engaging levels in Godot.



In this lesson, we will focus on implementing pickup items in our game. Pickup items are objects that the player can collect to enhance their abilities or gain new items, such as weapons or shields. We'll create a new scene for the pickup items, set up the logic for detecting collisions with the player, and handle the addition of items to the player's inventory. Additionally, we'll add some basic animations to make the pickup items more visually appealing.

Creating the Pickup Scene

To begin, we need to create a new scene for our pickup items. Follow these steps:

1. In Godot, create a new scene with a root node of type Node3D.
2. Rename the root node to Pickup.
3. Save the scene in a new folder named pickups within the scenes directory. Name the scene Pickup.tscn.

Setting Up the Pickup Item

Next, we'll set up the visual and collision aspects of the pickup item:

1. Add a MeshInstance3D node as a child of the Pickup node to display the pickup item visually.
2. Add an Area3D node as a child of the Pickup node to detect collisions with the player.
3. Add a CollisionShape3D node as a child of the Area3D node and set its shape to a SphereShape3D to define the collision area.

Adding the Pickup Script

To handle the logic for the pickup item, we need to add a script to the Pickup node:

1. Attach a new script to the Pickup node and name it Pickup.gd.
2. In the script, we'll define an enumeration for the different types of pickup items using the @export_enum directive. This allows us to select the type of pickup item from a dropdown menu in the Godot editor.
3. We'll also set up a variable to hold our item data.

Here is the initial setup of the Pickup.gd script:

```
extends Node3D
```

```
@export_enum("sword", "staff", "dagger", 'axe', 'square', 'round', 'spike', 'duckhat', 'tophat', 'sunglasses', 'starglasses') var pickup_type: String
var data: Dictionary
```

Loading Pickup Data

In the _ready function, we'll load the data for the pickup item based on its type. We'll use the dictionaries defined in the Global script to get the relevant data:

```
func _ready() -> void:
    for dict in [Global.weapons, Global.shields, Global.style]:
        if pickup_type in dict:
            data = dict[pickup_type]
            add_child(data['scene'].instantiate())
```



Testing the Pickup Item

To test our pickup item, we need to add it to the game scene:

1. Open the main game scene.
2. Add a new Node3D node and rename it to Pickups.
3. Add the Pickup scene as a child of the Pickups node.
4. Select the type of pickup item from the dropdown menu in the Godot editor.

Run the game and check if the pickup item is displayed correctly. You should see the selected item (e.g., a sword) in the game scene. Additionally, ensure that the data for the pickup item is printed to the console when the game runs.

Animating the Pickup Item

To make the pickup item more visually appealing, we'll add some basic animations. We'll rotate the item around the y-axis and move it up and down slightly using a sine wave:

```
func _physics_process(delta: float) -> void:
    rotate_y(2 * delta)
    position.y += sin(Time.get_ticks_msec() * delta * 0.2) * 0.03
```

Handling Player Collision

Finally, we'll handle the logic for when the player collides with the pickup item. We'll add the item to the player's inventory and destroy the pickup item:

1. Connect the `body_entered` signal of the Area3D node to a new function in the Pickup script.
2. In the connected function, check if the player has the item and, if not, add the pickup item's data to the player's inventory and destroy the pickup item.
3. We will also create a tween so that the pickup item has a nicer animation for disappearing before we remove it.

Here is the complete code for handling the player collision:

```
func _on_area_3d_body_entered(player: Node3D) -> void:
    var target = {'weapon': player.weapons, 'shield': player.shields, 'style': player
.styles}[data['type']]
    if data not in target:
        target.append(data)
    var tween = create_tween()
    tween.tween_property(self, 'scale', Vector3(0.1,0.1,0.1), 0.5)
    await tween.finished
    queue_free()
```

With these steps, you have successfully implemented pickup items in your game. The player can now collect weapons, shields, and style items, which will be added to their inventory. Additionally, the pickup items have basic animations to make them more visually appealing.



Welcome to this beginner-friendly guide on creating your first overworld level in Godot. In this tutorial, we will walk through the process of setting up your game scene, importing environment assets, and adding objects to your terrain. By the end, you will have a functional overworld level where your player can explore and interact with various elements.

Setting Up the Game Scene

Before we dive into creating the overworld, we need to set up our game scene properly. The game scene will serve as the parent for all level scenes, so it should contain only the essential elements.

1. Delete any existing terrain or unnecessary elements from the game scene.
2. Create a new Node3D and rename it to "Terrain". This node will contain all the environment elements for your levels.
3. Close any unnecessary scenes to keep your workspace clean. You should only have the game scene open.
4. Create a new inherited scene with the parent scene set to game.tscn. Rename this new scene to "overworld" and save it as overworld.tscn.

Importing the Overworld Environment

Now that our game scene is set up, we can import the overworld environment. This will serve as the main graphic for our overworld level.

1. Go to the graphics folder and navigate to the objects subfolder.
2. Locate the file named overworld_environment and double-click it to load it into Godot.
3. Ensure that the collision shapes are properly imported. You can check this by looking at the layers and ensuring that the terrain layer is set to layer one.
4. If there is a sun element in the environment, skip importing it as we do not need it for our project.
5. Reimport the entire overworld environment and drag it into the terrain node in your overworld scene.

Adding Objects to the Terrain

With the overworld environment in place, we can start adding objects to our terrain. Godot makes this process straightforward with its drag-and-drop functionality.

- Ensure you have the terrain node selected in your overworld scene.
- Browse the objects folder for elements like barrels, bridges, and trees.
- Drag and drop these objects into the terrain node to place them in your overworld.
- Test the collisions by moving the player character around the overworld.

Creating Collision Shapes

Some objects may not have collision shapes by default. In such cases, you will need to create them manually.

1. Select the object (e.g., a bridge) and go to its editable children.
2. Select the mesh and create a static body child.
3. Choose a collision shape (e.g., primesh) and create it.
4. Test the collision by running the scene and interacting with the object.

Populating the Overworld

The final step is to populate your overworld with various elements to make it visually appealing and



engaging for the player.

- Drag and drop trees, castles, and other objects from the objects folder into your terrain node.
- Use the “F” key to zoom in on specific objects for precise placement.
- Test your overworld by running the scene and exploring the environment.

Project Settings

To ensure that your game starts in the overworld scene, you need to adjust the project settings.

1. Go to Project > Project Settings.
2. Set the run scene to overworld instead of game.
3. Save your settings and run the game to start in the overworld scene.

Congratulations! You have successfully created your first overworld level in Godot.



In this part, we will walk you through the process of making your overworld more dynamic by adding water, which can later be reused for other liquids like lava. We'll use Godot's visual shader language to achieve this effect.

Setting Up the Water Mesh

1. Start by adding a MeshInstance3D node to your scene. Rename it to "Water".
2. Choose a plain mesh for the water surface. This mesh will be manipulated using a shader to create the water effect.
3. Position the mesh to cover the desired area. For example, you can set the size to 600 (width) and 200 (depth) to cover a canyon.
4. Increase the subdivision width and depth to add more vertices to the mesh. This is crucial for creating wave-like movements. Set the subdivision to 300 (width) and 100 (depth).

Creating the Shader

1. Create a new shader of type VisualShader and mode Spatial. Save it in the shader folder and name it "liquid".
2. Open the shader and switch to the vertex shader to start deforming the water mesh.

Deforming the Water Mesh

1. Get the vertex positions using the Vertex node.
2. Add a Vector3 node to manipulate the vertex positions in 3D space.
3. Create a Texture2D parameter named "wave_texture" for the noise texture.
4. Sample the texture using a Texture2D node and connect the sampler port.
5. Extract the red channel from the texture sample to use as the wave height.
6. Multiply the wave height with a float parameter to control the wave intensity.
7. Add the resulting value to the vertex positions to create the wave effect.

Adding Color to the Water

1. Switch to the fragment shader to add color to the water.
2. Create a VaryingSetter node to pass the wave texture data from the vertex shader to the fragment shader.
3. Use a VaryingGetter node in the fragment shader to retrieve the wave texture data.
4. Connect the wave texture data to the Albedo output to visualize the waves.
5. Create two color parameters, Color_A and Color_B, to color the waves.



6. Multiply Color_A with the wave texture data for the lighter portions of the waves.
7. Invert the wave texture data using a OneMinus node and multiply it with Color_B for the darker portions.
8. Combine the two colors using a Vector3Add node and connect it to the Albedo output.

Final Touches

- Adjust the colors and wave height to achieve the desired water effect.
- Experiment with different noise textures and parameters to create unique water surfaces.

Troubleshooting

- If the water effect is not visible, ensure that the mesh has enough vertices and the wave height is set appropriately.
- If the colors are not displaying correctly, check the connections between the nodes in the fragment shader.

With practice and experimentation, you'll become proficient in creating stunning visual effects using shaders in Godot. Happy learning!



Welcome back! In this part, we are going to animate the water and make it look more realistic. Currently, the water texture doesn't quite give the effect we want, so we'll need to work more on the shader and adjust the color to improve its appearance.

Adjusting the Noise Texture

The first step is to modify the noise texture to create a more water-like effect. We'll do this by adding a vector operation node to manipulate the texture.

- Add a vector operation node to the shader.
- Use the power operation to increase the contrast of the texture.

Adding Contrast Parameter

Next, we'll add a float parameter to control the contrast. This will allow us to adjust the intensity of the water effect dynamically.

- Add a float parameter named contrast.
- Set the range from 1 to 4, with a default value of 1.
- Connect this parameter to the power operation node.

Adjusting the Color

After adding the contrast, you might notice that the color isn't affected. This is because we need to update the varying parameter to use the modified texture.

- Update the varying parameter to use the texture with the power applied.
- Adjust the colors to make them more realistic.

Adding Alpha Value

To further enhance the water effect, we can add an alpha value to control the transparency.

- Add a float parameter named alpha.
- Set the range from 0 to 1, with a default value of 0.9.
- Connect this parameter to the alpha channel.

Animating the UV

Finally, we'll animate the UV to create a panning effect, making the water look like it's moving.

- Add a UVFunc node to the vertex shader.
- Add a time node to control the animation speed.
- Multiply the time node with a speed parameter to control the panning speed.
- Connect the result to the offset of the UVFunc node.

Adding Direction Parameter

To give more control over the animation direction, we'll add a Vector2 parameter.

- Add a Vector2 parameter named direction.
- Multiply this parameter with the speed to control the direction of the panning.
- Connect the result to the offset of the UVFunc node.

With these steps, you should have a more realistic water effect. Play around with the parameters to



get the look you want. Remember, there's no universal answer, so experimenting is key to achieving the best results.



In this lesson, we will focus on creating and animating a blade of grass in Godot. This process involves several steps, including setting up the grass blade mesh, creating a shader to animate it, and integrating it into the game world. Let's break down the steps:

Setting Up the Grass Blade Mesh

First, we need to create a new scene for the grass blade. This scene will be a `MeshInstance3D` node. Here's how to do it:

1. Create a new scene and add a `MeshInstance3D` node.
2. Rename the node to `GrassBlade`.
3. Save the scene in the `scenes/levels` folder.
4. Find the grass blade mesh (`grass-blade.obj`) in the `Graphics/Objects` folder and assign it to the `Mesh` property of the `MeshInstance3D` node.

Creating the Shader

Next, we will create a shader to animate the grass blade. This shader will make the grass blade move in the wind and away from the player.

1. Create a new `ShaderMaterial` and assign it to the `GrassBlade` node.
2. Create a new shader and set it to be a spatial shader.
3. Save the shader in the `shaders` folder.

Here is the initial shader code:

```
shader_type spatial;

void vertex() {

}

void fragment() {

}
```

Adjusting the Render Mode

To ensure the grass blade is visible from both sides, we need to adjust the render mode:

```
render_mode cull_disabled;
```

Defining Colors

We will define two colors: one for the grass blade and one for the shadow. Add the following lines to the shader:

```
uniform vec3 color : source_color;
uniform vec3 shadow_color : source_color;
```



In the shader parameters, set the color to a light green (00BB65) and the shadow_color to a neutral gray (666666).

Mixing Colors in the Fragment Shader

We will mix the colors in the fragment shader to create a gradient effect:

```
void fragment() {  
    ALBEDO = mix(color, shadow_color, UV.y);  
}
```

Adding Movement in the Vertex Shader

To add movement to the grass blade, we need to work in the vertex shader. First, we get the world position of the grass blade:

```
void vertex() {  
    vec2 world_pos = vec2(NODE_POSITION_WORLD.x, NODE_POSITION_WORLD.z);  
}
```

Creating a Curve Sampler

We will create a curve sampler to control the movement of the grass blade:

```
uniform sampler2D curve_sampler : repeat_disable;
```

In the shader parameters, define a curve texture and set the curve points to control the movement.

Sampling from the Curve

We will sample from the curve based on the UV coordinates:

```
float curve = texture(curve_sampler, vec2(UV.y, 0.0)).x;
```

Adding Wind Movement

To add random wind movement, we will use a noise texture:

```
uniform sampler2D wind_sampler : repeat_enable, filter_linear;
```



Sample from the noise texture based on the world position and time:

```
float wind = texture(wind_sampler, world_pos * 0.08 + TIME * 0.05).x - 0.5;
```

Combine the wind and curve values to move the grass blade:

```
VERTEX += vec3(wind * curve);
```

Integrating the Grass Blade into the Game World

Finally, we will integrate the grass blade into the game world:

1. Add a new Node3D to the terrain scene and rename it to Grass.
2. Add the GrassBlade scene as a child of the Grass node.
3. Position the grass blade close to the player for testing.

With these steps, you should have a blade of grass that moves in the wind. In the next lesson, we will further enhance our grass.



In this lesson, we will enhance the interactivity of our game by making the blades of grass move relative to the player's position. This will create a more immersive environment where the grass reacts to the player's presence. We'll achieve this by updating the shader to include the player's position and modifying the grass blade's vertex displacement accordingly.

Step 1: Adding the Player Position to Global Shader Parameters

To begin, we need to pass the player's position to the shader. This involves setting up a global shader parameter that will hold the player's position.

1. Open the **Project Settings**.
2. Navigate to the **Globals** tab.
3. Add a new global variable named `player_position` of type `Vector3`.

This global variable will allow us to access the player's position within the shader.

Step 2: Updating the Player Script

Next, we need to update the player script to set the global shader parameter with the player's position. This ensures that the shader always has the current position of the player.

1. Open the player script, which is typically found in the **player.gd** file.
2. Add the following line of code to update the global shader parameter:

```
RenderingServer.global_shader_parameter_set("player_position", global_position)
```

This line of code should be placed within the `_physics_process` function to ensure it updates every frame.

Step 3: Modifying the Grass Blade Shader

Now, we need to modify the grass blade shader to use the player's position for vertex displacement.

1. Open the grass blade shader.
2. Add a new uniform variable to hold the player's position:

```
global uniform vec3 player_position;
```

3. Update the vertex function to include the player displacement logic:

```
void vertex() {  
    ...  
  
    // Player displacement  
    vec3 direction_to_player = player_position - NODE_POSITION_WORLD;  
    float distance_to_player = length(direction_to_player);  
    VERTEX += -direction_to_player * falloff * curve * 1.5;  
}
```

This code calculates the direction and distance from the grass blade to the player and uses this information to displace the vertices of the grass blade away from the player.



Step 4: Adding Falloff to Limit the Effect

To ensure that the grass only moves when the player is close, we need to add a falloff effect. This will limit the displacement based on the distance to the player.

1. Update the vertex function to include the falloff calculation:

```
void vertex() {  
    ...  
  
    // Player displacement  
    vec3 direction_to_player = player_position - NODE_POSITION_WORLD;  
    float distance_to_player = length(direction_to_player);  
    float falloff = 1.0 - smoothstep(0.0, 1.0, distance_to_player);  
    VERTEX += -direction_to_player * falloff * curve * 1.5;  
}
```

The smoothstep function ensures that the falloff effect is smooth and gradual, creating a more natural movement of the grass blades.

Step 5: Testing and Refining

Finally, test the game to see the grass blades moving in response to the player's position. You may need to adjust the falloff and displacement values to achieve the desired effect.

- Run the game and observe the grass blades as the player moves around.
- Adjust the displacement and falloff values in the shader to refine the movement of the grass blades.

By following these steps, you will have successfully implemented interactive grass blades that move in response to the player's position, enhancing the immersive experience of your game.



In this tutorial, we will explore how to enhance your project's environment by scattering grass using the Proton Scatter plugin in Godot. This plugin is a powerful tool that not only scatters objects but also intelligently places them on surfaces, creating a more realistic and immersive scene.

Installing Proton Scatter

Before we can use Proton Scatter, we need to install it. Here are the steps to do so:

1. Open your project in Godot.
2. Go to the **Asset Library**.
3. Search for **Proton Scatter** and download it.
4. Once downloaded, click on **Install**.
5. After installation, enable the plugin by going to **Project -> Project Settings -> Plugins** and enabling **Proton Scatter**.

Understanding Proton Scatter

Proton Scatter works by using a Proton Scatter node, which has two children:

- **Scatter Item**: This is where you load the object you want to scatter, such as grass or trees.
- **Proton Shape**: This defines the area where the objects will be scattered.

Using Proton Scatter

To use Proton Scatter, follow these steps:

1. Add a **Proton Scatter** node to your scene.
2. Add a **Scatter Item** and a **Proton Shape** as children of the Proton Scatter node.
3. In the **Scatter Item**, load the object you want to scatter. For this tutorial, we will use a grass blade.
4. Adjust the **Proton Shape** to define the area where the grass will be scattered.

Customizing the Scatter

You can customize the scatter to fit your needs:

- Increase or decrease the number of objects scattered by adjusting the **Create Inside** property in the Proton Scatter node.
- Change the shape and size of the scatter area by adjusting the properties in the Proton Shape node.
- Duplicate the Proton Scatter node to cover larger areas with grass.

Optimizing Performance

Scattering a large number of objects can impact performance. Here are some tips to optimize:

- Only scatter objects in visible areas.
- Adjust the scatter shape and size to fit the specific area you want to cover.
- Experiment with different shapes and sizes to find the best balance between visual quality and performance.

By following this tutorial, you should now have a good understanding of how to use the Proton Scatter plugin in Godot to enhance your project's environment. Don't be afraid to experiment with different settings and objects to create the perfect scene for your game.



Welcome to this beginner-friendly guide on creating a castle level in Godot using a grid map. In this tutorial, we will walk you through the process of setting up your castle scene, understanding grid maps, and designing your level with various elements. By the end of this article, you will have a basic castle level ready for exploration.

Getting Started

Before we dive into the details, ensure you have Godot open and your project ready. We will be creating a new scene for our castle level.

Creating the Castle Scene

1. In Godot, create a new inherited scene with `game.tscn` as the parent scene.
2. Rename the new scene to `castle` and save it inside the `levels` folder.

Setting Up the Grid Map

A grid map allows you to create complex levels dynamically. Here's how to set it up:

1. Create a grid map node in your castle scene.
2. Initially, the grid map will be empty. To populate it, you need a mesh library.
3. Go to the quick load menu and select `dungeon_grid_map`. This will provide you with various elements to place in your grid map.

Placing Elements in the Grid Map

With the grid map set up, you can start placing elements:

- Select the paintbrush tool in the grid map.
- Choose an element from the mesh library and place it in the grid map. For example, place a floor tile below the player.
- Ensure the player is positioned above the placed elements.

Running the Scene

After placing the initial elements, run the scene to ensure everything is working correctly. You should be able to move around in the basic scene you've created.

Creating the Dungeon Grid Map

To create the dungeon grid map:

1. Navigate to the `graphics` folder and open `DungeonTiles.tscn`.
2. Inside this scene, you will find various nodes, each with a mesh, static body, and collision shape.
3. These nodes follow a consistent pattern, making it easy to add new elements.

Exporting as a Mesh Library

Once you have all your elements set up:

1. Go to the scene menu and select `Export as Mesh Library`.
2. This will allow you to import all elements into your grid map.

Adjusting Cell Size



To avoid overlaps between items:

- Adjust the cell size in the grid map settings.
- Set the x and z values to 4 to ensure each grid cell contains one item.

Creating Floor and Wall Grid Maps

For better organization:

1. Create two grid maps: one for the floor and one for the walls.
2. Duplicate the initial grid map to create the wall grid map.
3. In the wall grid map, remove all floor tiles to ensure no overlaps.

Designing the Castle Level

With the grid maps set up, you can start designing your castle level:

- Use the paintbrush tool to place floor tiles and walls.
- Rotate objects using the ASD keys to position them accurately.
- Add additional objects from the objects folder to enhance your level.

Adding Staircases and Other Objects

To make your level more interesting:

- Add staircases as separate objects to allow scaling on the x-axis.
- Include a static body node for collisions with the stairs.
- Add other objects like skeletons and pickups to make the level engaging.

Final Touches

Once your level is designed:

- Run the scene to test collisions and interactions.
- Ensure all elements are correctly placed and functional.

Conclusion

Congratulations! You have successfully created a basic castle level in Godot using a grid map. This level can be further enhanced with moving platforms, lava, and other interactive elements. Stay tuned for more advanced tutorials on Zenva.

Happy game development!



In this lesson, we will enhance our castle level by introducing moving platforms. These platforms will add a dynamic element to the game, making it more engaging for players. We will use Godot's `AnimatableBody3D` node to create these moving platforms. This node allows us to move static bodies, which is perfect for our needs.

Creating the Moving Platform Scene

To start, we need to create a new scene for our moving platform:

1. Create a new scene and set the root node to `AnimatableBody3D`.
2. Rename the root node to `MovingPlatform`.
3. Save the scene as `moving_platform.tscn` in the `scenes/levels` directory.

Adding a Mesh and Collision Shape

Next, we need to add a mesh and a collision shape to our moving platform so that it is visible and can interact with other objects in the game:

1. Add a mesh to the `MovingPlatform` node. For this lesson, we will use the `floor_tile_big_crate` from the `dungeon_gridmap`.
2. To add the mesh, open the `dungeon_gridmap` and select the `floor_tile_big_grate_open2`. Then, attach it to the `MovingPlatform` scene.
3. Enable editable children and click on the mesh to create a collision shape.
4. Select `Create Trimesh Static Body` and then move the created `CollisionShape3D` as a child of the `MovingPlatform` node.

Scripting the Moving Platform

Now, we need to add a script to control the movement of the platform:

1. Attach a new script to the `MovingPlatform` node called **`moving_platform.gd`**.
2. Add the following code to the script. This code will store a direction and speed, and then move the platform along the x axis over time:

```
extends AnimatableBody3D

var direction = 1
@export var speed: float = 4.0

func _physics_process(delta: float) -> void:
    position.x += speed * direction * delta
```

Adding the Moving Platform to the Castle Scene

With the moving platform scene created, we can now add it to our castle scene:

1. Open the castle scene.
2. Add the `MovingPlatform` scene to the terrain.
3. Position the moving platform where you want it to be in the level.

Making the Platform Move Between Points

To make the platform move between two points, we need to add some logic to reverse the direction



when the platform reaches a certain point:

1. Add an Area3D node as a child of the MovingPlatform node.
2. Add a CollisionShape3D as a child of the Area3D node and set its shape to a BoxShape3D.
3. Resize the BoxShape3D to cover an area larger than the platform itself.
4. Connect the body_entered signal of the Area3D node to the script.
5. Add the following code to the script to handle the direction reversal:

```
func _on_area_3d_body_entered(body: Node3D) -> void:
    if body != self:
        direction *= -1
```

Testing the Moving Platform

With the script and collision shape in place, you can now test the moving platform in the castle scene. The platform should move between the two points and reverse direction when it collides with the walls.

Duplicating the Moving Platform

To add more moving platforms to the level, you can duplicate the existing platform and position the duplicates as needed. This will create a more challenging and dynamic environment for the player.

Replacing the Blue Bit with Lava

Finally, to make the level look more appealing, we will replace the blue bit with lava:

1. Add another MeshInstance to the scene and move it to the right position.
2. Set the mesh to a BoxMesh and size it up to cover the entire available area.
3. Flip the faces of the BoxMesh so that only the inside is visible.
4. Assign a standard material to the BoxMesh and set its color to black.
5. Add another MeshInstance with a PlaneMesh and position it inside the box.
6. Assign a shader material to the PlaneMesh and load the liquid shader.
7. Set the textures and fast noise lite for the shader.
8. Adjust the colors and other properties of the shader to achieve the desired lava effect.

By following these steps, you will have created moving platforms that add a dynamic element to your castle level. Additionally, replacing the blue bit with lava will enhance the visual appeal of the level.



In this lesson, we will focus on connecting two basic levels in our game and enabling the player to transition between them. Specifically, we will work on loading different scenes and adding a fade-to-black animation to make the transition smoother. We will use Godot's scene management and tweening capabilities to achieve this.

Setting Up the Scene Structure

To begin, we need to ensure our scene structure is organized correctly. We will use the following scenes:

- game.tscn: The main game scene.
- overworld.tscn: The overworld level scene.
- castle.tscn: The castle level scene.

Creating the Transition Gate

We will create a new node in the game.tscn scene to manage the transition between levels. Follow these steps:

1. Add a new Node3D to the game.tscn scene and rename it to TransitionGates.
2. Add a new Area3D as a child of TransitionGates and rename it to TransitionGate.
3. Save the TransitionGate scene in the levels/transition folder.

Configuring the Transition Gate

Next, we need to configure the TransitionGate to detect collisions with the player. Follow these steps:

1. Add a CollisionShape3D to the TransitionGate and set its shape to a SphereShape3D with a radius of 1.5.
2. Set the collision layer to No Layer and the collision mask to Layer 2 (the player layer). This ensures that only the player can interact with the transition gate.

Adding the Transition Script

We will now add a script to the TransitionGate to handle the scene transition. Create a new script and attach it to the TransitionGate. In this script, we need to create an Enum of our levels and save what scenes those levels are associated. Then, connecting up the **_on_body_entered signal**, we can change the scene.

```
extends Area3D
```

```
@export_enum('Castle', 'Overworld') var target = 'Overworld'
var targets = {
    'Castle': "res://scenes/levels/castle.tscn",
    'Overworld': "res://scenes/levels/overworld.tscn"
}
```

```
func _on_body_entered(_body: Node3D) -> void:
    TransitionLayer.change_scene(targets[target])
```

Creating the Transition Layer



To create a smooth fade-to-black transition, we need to create a new scene with a CanvasLayer as the root node. Follow these steps:

1. Create a new scene and add a CanvasLayer as the root node. Rename the scene to TransitionLayer.
2. Add a ColorRect to the CanvasLayer and set its color to black. Ensure it covers the entire screen.
3. Save the TransitionLayer scene in the levels/transition folder.

Adding the Transition Layer Script

We will now add a script to the TransitionLayer to handle the fade-to-black animation. Create a new script and attach it to the TransitionLayer. Using the ColorRect, we will create a tween that handles the fading in and out process. We'll also create a function that actually changes the scene. Using a callback for the tween, we can call this function so the scene loads at the proper time.

```
extends CanvasLayer
```

```
func _ready():  
    $ColorRect.modulate = Color(0,0)
```

```
func change_scene(target):  
    var tween = create_tween()  
    tween.tween_property($ColorRect, 'modulate', Color(0,1), 0.5)  
    tween.tween_callback(Callable(self, 'open_scene').bind(target))  
    tween.tween_property($ColorRect, 'modulate', Color(0,0), 0.5)
```

```
func open_scene(scene_path):  
    get_tree().change_scene_to_file(scene_path)
```

Making the Transition Layer Global

To ensure the transition layer is available throughout the game, we need to make it global. Follow these steps:

1. Open the project settings and go to the Globals tab.
2. Add the TransitionLayer scene to the globals.

Testing the Transition

Finally, we need to test the transition to ensure it works correctly. Follow these steps:

1. Add a TransitionGate to the castle.tscn scene near the entrance.
2. Set the target property of the TransitionGate to Overworld.
3. Run the game and move the player towards the transition gate to trigger the scene transition.

By following these steps, you should be able to create a smooth transition between the overworld and castle levels in your game. The fade-to-black animation will make the transition look more polished and less abrupt.



Congratulations on completing your course! By this point, you have gained a significant amount of knowledge and skills. Throughout this course, you have learned to create levels in two different ways—via Blender import and via a grid map. Additionally, you have used shaders to enhance the visual appeal of your project. Overall, you have constructed a comprehensive project with numerous moving parts.

Key Achievements

- Created levels using Blender import and grid map techniques.
- Applied shaders to improve the visual quality of the project.
- Developed a large project with multiple components.

Expanding Your Project

Now that you have a solid foundation, you can expand your project in various ways. Here are some recommendations to further enhance your 3D game:

Adding Enemies and Pickups

To make your game more engaging, consider adding enemies and pickups throughout the levels. You already have nodes for characters and pickups, making it easy to integrate these elements.

1. Create enemy and pickup items.
2. Set their positions within the levels.

Creating More Levels

Expand your game by creating additional levels and enhancing the transition system. This will allow players to travel to more spots within the game.

- Design new levels such as caves or water levels.
- Improve the transition system to load scenes with the player in different starting positions.

Introducing a Boss Enemy

Adding a boss enemy can provide a challenging and exciting experience for players. This is an excellent exercise to gain more experience with enemy design.

Incorporating Magic Effects

Enhance your game with magical effects like fireballs. These visual elements can make the gameplay more dynamic and visually appealing.

Utilizing Asset Packs

The asset pack used in this project is called *Cakeit Dungeon Pack*, which is entirely free. This pack includes numerous art assets that you can integrate into your project to improve its visual quality.

- Explore the asset pack for additional art assets.
- Incorporate these assets to enhance the overall look of your game.

We hope you enjoyed this series and found it valuable. Keep practicing and exploring new ideas to further develop your skills. See you around!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

global.gd

Found in folder: **/Global**

This code defines a set of dictionaries that store data for various game items, including weapons, shields, and style items. Each item has properties such as damage, defense, thumbnails, and scenes associated with it. The code also includes a function to pause and unpauses the game, which reveals or hides the inventory when paused.

extends Node

```
@onready var inventory = get_tree().get_first_node_in_group("Inventory")
var weapons: Dictionary = {
    'dagger':
    {
        'type': 'weapon',
        'damage': 1,
        'thumbnail': preload("res://graphics/thumbnails/dagger.png"),
        'scene': preload("res://scenes/weapons/dagger.tscn"),
        'animation': '1H_Melee_Attack_Stab',
        'range': 1.2,
        'audio': preload("res://audio/dagger_sound.wav")
    },
    'sword':
    {
        'type': 'weapon',
        'damage': 2,
        'thumbnail': preload("res://graphics/thumbnails/sword.png"),
        'scene': preload("res://scenes/weapons/sword.tscn"),
        'animation': '1H_Melee_Attack_Slice_Horizontal',
        'range': 1.5,
        'audio': preload("res://audio/sword_sound.wav")
    },
    'axe':
    {
        'type': 'weapon',
        'damage': 3,
        'thumbnail': preload("res://graphics/thumbnails/axe.png"),
        'scene': preload("res://scenes/weapons/axe.tscn"),
        'animation': '2H_Melee_Attack_Spin',
        'range': 1.3,
        'audio': preload("res://audio/axe_sound.wav")
    },
    'staff':
    {
        'type': 'weapon',
        'damage': 1,
        'thumbnail': preload("res://graphics/thumbnails/staff.png"),
```

```
'scene': preload("res://scenes/weapons/staff.tscn"),
'animation': '2H_Melee_Attack_Slice',
'range': 2.1,
'audio': preload("res://audio/staff_sound.wav")
},
}
var shields: Dictionary = {
'square': {
'type': 'shield',
'defense': 0.8,
'thumbnail': preload("res://graphics/thumbnails/square.png"),
'scene': preload("res://scenes/shields/square_shield.tscn")
},
'round': {
'type': 'shield',
'defense': 0.9,
'thumbnail': preload("res://graphics/thumbnails/round.png"),
'scene': preload("res://scenes/shields/round_shield.tscn")
},
'spike': {
'type': 'shield',
'defense': 0.6,
'thumbnail': preload("res://graphics/thumbnails/spike.png"),
'scene': preload("res://scenes/shields/spike_shield.tscn")
},
}
var style: Dictionary = {
'sunglasses': {
'type': 'style',
'scene': preload("res://scenes/style/sunglasses.tscn"),
'thumbnail': preload("res://graphics/thumbnails/sun_glasses.png")
},
'starglasses': {
'type': 'style',
'scene': preload("res://scenes/style/starglasses.tscn"),
'thumbnail': preload("res://graphics/thumbnails/star_glasses.png")
},
'duckhat': {
'type': 'style',
'scene': preload("res://scenes/style/duck_hat.tscn"),
'thumbnail': preload("res://graphics/thumbnails/duck.png")
},
'tophat': {
'type': 'style',
'scene': preload("res://scenes/style/tophat.tscn"),
'thumbnail': preload("res://graphics/thumbnails/top_hat.png")
},
}
}

func pause(value: bool = true):
if $PauseTimer.time_left == 0.0:
get_tree().paused = value
$PauseTimer.start()
if value:
```

```
inventory.reveal()
else:
    inventory.hide()
```

camera_controller.gd

Found in folder: **/scenes/characters**

This script controls the rotation of a 3D camera arm based on user input. It uses joystick or gamepad input to rotate the camera, with adjustable acceleration and limits on the rotation. The script also includes commented-out code for capturing mouse input and rotating the camera based on mouse movement.

```
extends SpringArm3D
```

```
@export var min_limit_x: float = -0.8
@export var max_limit_x: float = 0.2
@export var horizontal_acceleration: float = 2
@export var vertical_acceleration: float = 1
@export var mouse_acceleration: float = 0.005

func _process(delta: float) -> void:
    var joy_dir = Input.get_vector("pan_left", "pan_right", "pan_up", "pan_down")
    var joy_dir_accelerated = joy_dir * delta * Vector2(horizontal_acceleration, vertical_acceleration)
    rotate_from_vector(joy_dir_accelerated)

#func _ready() -> void:
#    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
#
#func _input(event: InputEvent) -> void:
#    #if event is InputEventMouseMotion:
#        #var mouse_dir_accelerated = event.relative * mouse_acceleration
#        #rotate_from_vector(mouse_dir_accelerated)
#    #if Input.is_action_just_pressed("ui_cancel"):
#        #get_tree().quit()

func rotate_from_vector(v: Vector2):
    if v.length() == 0 : return
    rotation.y -= v.x
    rotation.x -= v.y
    rotation.x = clamp(rotation.x, min_limit_x, max_limit_x)
```

character.gd

Found in folder: **/scenes/characters**

This script defines the behavior of a character in a 3D game, including movement, jumping, attacking, defending, and health management. It also handles animation states, equipment management, and collision detection. The character's properties, such as speed, jump height, and health, can be adjusted through exported variables.



```
class_name Character
extends CharacterBody3D

@export var base_speed: float = 4
@export var run_speed: float = 6
@export var defend_speed: float = 2
var movement_input: Vector2

@export var jump_height: float = 2.25
@export var jump_time_to_peak: float = 0.4
@export var jump_time_to_descent: float = 0.3

@onready var jump_velocity: float = ((2.0 * jump_height) / jump_time_to_peak) * -1.0
@onready var jump_gravity: float = ((-2.0 * jump_height) / (jump_time_to_peak * jump_time_to_peak)) * -1.0
@onready var fall_gravity: float = ((-2.0 * jump_height) / (jump_time_to_descent * jump_time_to_descent)) * -1.0

@onready var move_state_machine = $AnimationTree.get("parameters/MoveStateMachine/playback") as AnimationNodeStateMachinePlayback
@onready var attack_animation = $AnimationTree.get_tree_root().get_node('AttackAnimation') as AnimationNodeAnimation

var skin: Node3D
var attacking: bool = false
var defending: bool = false:
    set(value):
        if not defending and value:
            defend_toggle(true)
        if defending and not value:
            defend_toggle(false)
        defending = value
var current_weapon: Node3D
var current_shield: Node3D
var squash_and_stretch: float = 1.0:
    set(value):
        squash_and_stretch = value
        var negative = 1.0 + (1.0 - squash_and_stretch)
        skin.scale = Vector3(negative, squash_and_stretch, negative)
var health: int = 3:
    set(value):
        if "hud" in self:
            get_tree().get_first_node_in_group("HUD").set_health(value)
        if value <= 0:
            get_tree().quit()
        health = value

func defend_toggle(forward: bool):
    var tween = create_tween()
    tween.tween_method(_defend_change, 1.0 - float(forward), float(forward), 0.25)

func _defend_change(value):
    $AnimationTree.set("parameters/DefendBlend/blend_amount", value)
```

```
func apply_gravity(gravity, delta):
    velocity.y -= gravity * delta

func set_move_state(state_name: String):
    move_state_machine.travel(state_name)

func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
        current_weapon = item_scene
        current_weapon.set_sound(data['audio'])
    if data['type'] == 'shield':
        item_scene.defense = data['defense']
        current_shield = item_scene

func _on_animation_tree_animation_finished(anim_name: StringName) -> void:
    if 'Attack' in anim_name:
        attacking = false

func attack_logic():
    if attacking:
        var collider = current_weapon.get_collider()
        if collider and collider != self and "hit" in collider:
            collider.hit(current_weapon)

func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            health -= attacking_weapon.damage * current_shield.defense
            current_shield.flash()
            current_shield.play_audio()
        else:
            health -= attacking_weapon.damage
            $HitSound.play()
            $Timers/HitTimer.start()
            do_squash_and_stretch(1.2, 0.2)

    if health <= 0:
        death_logic()

func death_logic():
    pass
```

```
func do_squash_and_stretch(value: float, duration: float):
    var tween = create_tween()
    tween.tween_property(self, "squash_and_stretch", value, duration)
    tween.tween_property(self, "squash_and_stretch", 1.0, duration * 1.8).set_ease(Tween.EASE_OUT)
```

player.gd

Found in folder: **/scenes/characters**

This script controls the player character's movement, actions, and equipment. It handles input for movement, jumping, attacking, and switching between weapons and shields. The script also updates the player's animation state and applies gravity and movement physics.

extends Character

```
@export var acceleration: float = 8
@export var deceleration: float = 4
@onready var camera = $CameraController/Camera3D
@onready var hud = get_tree().get_first_node_in_group("HUD")

var weapons = [Global.weapons['sword'], Global.weapons['staff']]
var weapon_index: int
var shields = [Global.shields['round'], Global.shields['square'], Global.shields['spike']]
var shield_index: int
var styles = [Global.style['duckhat']]
var style_index: int = -1

func _ready() -> void:
    skin = $PlayerSkin
    equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
    #equip(styles[style_index], $PlayerSkin/Rogue/Rig/Skeleton3D/Head)
    hud.setup(health)

func _physics_process(delta: float) -> void:
    RenderingServer.global_shader_parameter_set("player_position", global_position)
    move_logic(delta)
    jump_logic(delta)
    ability_logic()
    move_and_slide()
    attack_logic()

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("switch_weapon"):
        weapon_index = posmod(weapon_index + 1, weapons.size())
        equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    if Input.is_action_just_pressed("switch_shield"):
        shield_index = posmod(shield_index + 1, shields.size())
```



```
equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
if Input.is_action_just_pressed("ui_focus_next"):
    health += 1
if Input.is_action_just_pressed("ui_cancel"):
    health -= 1
if Input.is_action_just_pressed("menu"):
    Global.pause()

func move_logic(delta):
    movement_input = Input.get_vector("left", "right", "forward", "backward").rotated(-camera.global_rotation.y)
    var vel_2d = Vector2(velocity.x, velocity.z)

    if movement_input != Vector2.ZERO:
        var speed = run_speed if Input.is_action_pressed("run") else base_speed
        speed = defend_speed if defending else speed
        vel_2d += movement_input * speed * delta * acceleration
        vel_2d = vel_2d.limit_length(speed)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        var target_angle = -movement_input.angle() + PI / 2
        skin.rotation.y = rotate_toward(skin.rotation.y, target_angle, delta * 6)
        set_move_state('Running_A')
    else:
        vel_2d = vel_2d.move_toward(Vector2.ZERO, base_speed * delta * deceleration)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        set_move_state('Idle')

func jump_logic(delta):
    if is_on_floor():
        if Input.is_action_just_pressed("jump"):
            velocity.y = -jump_velocity
    else:
        set_move_state('Jump_Idle')
    var gravity = jump_gravity if velocity.y > 0.0 else fall_gravity
    apply_gravity(gravity, delta)

func ability_logic():
    defending = Input.is_action_pressed("defend")
    if Input.is_action_just_pressed("attack"):
        if not attacking:
            $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
            attacking = true
            current_weapon.play_audio()

func death_logic():
    get_tree().quit()
```

```
func unequip():  
    for child in $PlayerSkin/Rogue/Rig/Skeleton3D/Head.get_children():  
        child.queue_free()
```

skeleton.gd

Found in folder: **/scenes/characters**

This code controls an enemy character that moves towards the player and attacks when within a certain range. The enemy's movement and attack logic are updated every physics frame, and it uses animations and audio effects to enhance the gameplay experience. When the enemy's health reaches zero, it triggers a death animation and disables its collision shape.

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')  
@export var notice_radius: float = 30  
@export var speed: float = 3  
var attack_radius: float = 3  
var rng = RandomNumberGenerator.new()  
  
func _ready() -> void:  
    for child in $Skins.get_children():  
        child.hide()  
    skin = $Skins.get_children().pick_random()  
    skin.show()  
    $AnimationTree.anim_player = "../Skins/" + skin.name + '/AnimationPlayer'  
    var random_weapon = Global.weapons[Global.weapons.keys().pick_random()]  
    equip(random_weapon, skin.get_node('Rig/Skeleton3D/RightHand'))  
    attack_radius = current_weapon.radius  
    health = 3
```

```
func _physics_process(delta: float) -> void:  
    if health > 0:  
        move_to_player(delta)  
        if not is_on_floor():  
            apply_gravity(fall_gravity, delta)  
        else:  
            velocity.y = 0  
        move_and_slide()  
        attack_logic()
```

```
func move_to_player(delta):  
    if position.distance_to(player.position) < notice_radius:  
        var target_dir = (player.position - position).normalized()  
        var target_vec2 = Vector2(target_dir.x, target_dir.z)  
        var target_angle = -target_vec2.angle() + PI/2  
        rotation.y = rotate_toward(rotation.y, target_angle, delta * 6)  
    if position.distance_to(player.position) > attack_radius:  
        velocity = Vector3(target_vec2.x, 0, target_vec2.y) * speed
```



```
        set_move_state('Running_A')
    else:
        velocity = Vector3.ZERO
        set_move_state('Idle')

func _on_attack_timer_timeout() -> void:
    $Timers/AttackTimer.wait_time = rng.randf_range(2.0, 3.5)
    if position.distance_to(player.position) < attack_radius:
        $AnimationTree.set('parameters/AttackOneShot/request', AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
        attacking = true
        current_weapon.play_audio()

func death_logic():
    $CollisionShape3D.disabled = true
    $Timers/AttackTimer.stop()
    var tween = create_tween()
    tween.tween_method(_death_change, 0.0, 1.0, 0.25)

func _death_change(value):
    $AnimationTree.set("parameters/DeathBlend/blend_amount", value)
```

moving_platform.gd

Found in folder: **/scenes/levels**

This code moves an object along the x-axis at a specified speed. The direction of movement is reversed when the object collides with another object. The speed of movement can be adjusted using the exported `speed` variable.

```
extends AnimatableBody3D

var direction = 1
@export var speed: float = 4.0

func _physics_process(delta: float) -> void:
    position.x += speed * direction * delta

func _on_area_3d_body_entered(body: Node3D) -> void:
    if body != self:
        direction *= -1
```

transition_gate.gd

Found in folder: **/scenes/levels/transition**

This code defines a 3D area that triggers a scene change when a body enters it. The target scene is determined by the `target` variable, which can be set to either "Castle" or "Overworld". When a

body enters the area, the scene is changed to the corresponding scene path defined in the `targets` dictionary.

```
extends Area3D
```

```
@export_enum('Castle', 'Overworld') var target = 'Overworld'
var targets = {
    'Castle': "res://scenes/levels/castle.tscn",
    'Overworld': "res://scenes/levels/overworld.tscn"
}
```

```
func _on_body_entered(_body: Node3D) -> void:
    TransitionLayer.change_scene(targets[target])
```

transition_layer.gd

Found in folder: **/scenes/levels/transition**

This code controls a transition effect between scenes. When the `change\_scene` function is called, it fades in a black rectangle, changes to the target scene, and then fades the rectangle back out. The `open\_scene` function is used to actually switch to the new scene.

```
extends CanvasLayer
```

```
func _ready():
    $ColorRect.modulate = Color(0,0)

func change_scene(target):
    var tween = create_tween()
    tween.tween_property($ColorRect, 'modulate', Color(0,1), 0.5)
    tween.tween_callback(Callable(self, 'open_scene').bind(target))
    tween.tween_property($ColorRect, 'modulate', Color(0,0), 0.5)
```

```
func open_scene(scene_path):
    get_tree().change_scene_to_file(scene_path)
```

pickup.gd

Found in folder: **/scenes/pickups**

This script defines a 3D node that represents a pickup item. When the node is initialized, it loads data from a dictionary based on its pickup type and instantiates a corresponding scene as a child. When a player collides with the node, it adds the pickup item's data to the player's inventory and animates the node's disappearance.

```
extends Node3D
```

```
@export_enum("sword", "staff", "dagger", 'axe', 'square', 'round', 'spike', 'duckhat')
```



```
, 'tophat', 'sunglasses', 'starglasses') var pickup_type: String
var data: Dictionary

func _ready() -> void:
    for dict in [Global.weapons, Global.shields, Global.style]:
        if pickup_type in dict:
            data = dict[pickup_type]
            add_child(data['scene'].instantiate())

func _physics_process(delta: float) -> void:
    rotate_y(2 * delta)
    position.y += sin(Time.get_ticks_msec() * delta * 0.2) * 0.03

func _on_area_3d_body_entered(player: Node3D) -> void:
    var target = {'weapon': player.weapons, 'shield': player.shields, 'style': player.styles}[data['type']]
    if data not in target:
        target.append(data)
    var tween = create_tween()
    tween.tween_property(self, 'scale', Vector3(0.1,0.1,0.1), 0.5)
    await tween.finished
    queue_free()
```

shield.gd

Found in folder: **/scenes/shields**

This script extends the Node3D class and represents a shield equipment with a defense value. It has a flash function that creates a tween to temporarily change the alpha value of a material overlay, causing a flashing effect. The script also has a play_audio function that plays a sound effect when called.

```
extends Node3D
```

```
var defense: float
const equipment_type = 'Shield'
```

```
func flash():
    var tween = create_tween()
    tween.tween_method(_flash, 0.0, 1.0, 0.2)
    tween.tween_method(_flash, 1.0, 0.0, 0.4)
```

```
func _flash(value):
    get_child(0).get_child(0).material_overlay.set_shader_parameter('alpha', value)
```

```
func play_audio():
    $HitSound.play()
```

hud.gd

Found in folder: **/scenes/ui**

This code creates a control that displays a set of hearts, where the number of hearts is determined by the `setup` function. The `setup` function instantiates a specified number of heart scenes and adds them as children to the `$Hearts` node. The `set_health` function clears any existing hearts and then calls `setup` to display the new number of hearts.

```
extends Control

var heart_scene = preload("res://scenes/ui/heart.tscn")

func setup(hearts: int):
    for heart in hearts:
        var heart_instance = heart_scene.instantiate()
        $Hearts.add_child(heart_instance)

func set_health(health: int):
    for child in $Hearts.get_children():
        child.queue_free()
    setup(health)
```

inventory.gd

Found in folder: **/scenes/ui**

This code manages the player's inventory and equipment selection. It populates grid containers with equipment items, handles user input to switch between equipment categories and select items, and updates the player preview accordingly. The code also controls the rotation of the player preview based on user input.

```
extends Control

@onready var player_preview = get_tree().get_first_node_in_group("PlayerPreview")
@onready var player = get_tree().get_first_node_in_group("Player")
@onready var item_scene = preload("res://scenes/ui/item.tscn")
@export var pan_speed:float = 3

@onready var weapon_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Weapons
@onready var shield_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Shields
@onready var style_grid = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer/Style
@onready var tab_container = $HBoxContainer/ItemPanelContainer/MarginContainer2/TabContainer
const border_colors = [Color.LIME_GREEN, Color.FIREBRICK, Color.GOLDENROD]

func reveal():
    for child in tab_container.get_children():
        for node in child.get_children():
```

```
node.queue_free()
show()
create_inventory_items(weapon_grid, player.weapons, player.weapon_index)
create_inventory_items(shield_grid, player.shields, player.shield_index)
create_inventory_items(style_grid, player.styles, player.style_index)
focus()

func focus():
    await get_tree().create_timer(0.01).timeout
    if tab_container.get_child(tab_container.current_tab).get_child_count():
        tab_container.get_child(tab_container.current_tab).get_child(0).grab_focus()

func _process(delta: float) -> void:
    var pan_dir = Input.get_axis("pan_left", "pan_right")
    player_preview.rotate_y(pan_dir * delta * pan_speed)

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("menu"):
        Global.pause(false)
        var menu_dir = int(Input.get_axis("switch_shield", "switch_weapon"))
        if menu_dir:
            tab_container.current_tab = posmod(tab_container.current_tab + menu_dir, tab_container.get_child_count())
            focus()
            $HBoxContainer/ItemPanelContainer/Control/Label.text = tab_container.get_child(tab_container.current_tab).name
            var color = border_colors[tab_container.current_tab]
            $HBoxContainer/ItemPanelContainer/MarginContainer/Panel.get_theme_stylebox("panel").border_color = color
            $HBoxContainer/ItemPanelContainer/Control/Label.add_theme_color_override("font_color", color)
            if Input.is_action_just_pressed("run"):
                if tab_container.current_tab == 2:
                    player_preview.unequip()
                    player.unequip()

func create_inventory_items(container: GridContainer, equipment_list: Array, index: int):
    for i in equipment_list.size():
        var equipment_item = equipment_list[i]
        var item_instance = item_scene.instantiate()
        item_instance.setup(equipment_item)
        container.add_child(item_instance)
        item_instance.highlight(i == index)
        if i == index:
            player_preview.add_equipment(equipment_item)
```

item.gd

Found in folder: **/scenes/ui**



This script extends the Button class and is used to display and manage equipment items. It sets up the button's appearance and behavior based on the provided equipment data, and handles user interactions such as highlighting and equipping the item. When pressed, the button updates the player's equipment and preview, and highlights itself as the currently equipped item.

```
extends Button
```

```
@onready var player_preview = get_tree().get_first_node_in_group("PlayerPreview")
@onready var player = get_tree().get_first_node_in_group("Player")
var equipment_data
```

```
func _ready() -> void:
    size_flags_horizontal = Control.SIZE_EXPAND | Control.SIZE_FILL
```

```
func setup(data):
    $TextureRect.texture = data['thumbnail']
    equipment_data = data
```

```
func _on_resized() -> void:
    custom_minimum_size.y = get_rect().size.x
```

```
func highlight(value: bool):
    if value:
        $EquippedPanel.show()
    else:
        $EquippedPanel.hide()
```

```
func _on_pressed() -> void:
    # update the real player
    var slot = {'weapon': 'RightHand', 'shield': 'LeftHand', 'style': 'Head'}[equipment_data['type']]
    player.equip(equipment_data, player.get_node('PlayerSkin/Rogue/Rig/Skeleton3D/' + slot))

    #update the preview
    player_preview.add_equipment(equipment_data)
    for item in get_parent().get_children():
        item.highlight(false)
    highlight(true)
```

rogue.gd

Found in folder: **/scenes/ui**

This code adds equipment to a 3D character model by instantiating a scene and attaching it to a specific slot on the character's rig. The slot is determined by the type of equipment being added, such as a weapon, shield, or style item. The code also provides a function to unequip items from the character's head slot, removing any child nodes from that location.

```
extends Node3D
```

```
func add_equipment(data):
    var slot = {
        'weapon': $Rig/Skeleton3D/RightHand,
        'shield': $Rig/Skeleton3D/LeftHand,
        'style': $Rig/Skeleton3D/Head
    }[data['type']]
    for child in slot.get_children():
        child.queue_free()
    var item_instance = data['scene'].instantiate()
    slot.add_child(item_instance)

func unequip():
    for child in $Rig/Skeleton3D/Head.get_children():
        child.queue_free()
```

weapon.gd

Found in folder: **/scenes/weapons**

This script defines a 3D node that represents a weapon in the game. It stores properties such as animation, damage, and radius, which are set up through the `setup` function. The script also provides methods to get the collider, set and play audio for the weapon's attack sound.

```
extends Node3D
```

```
const equipment_type = 'Weapon'
var animation: String
var damage: int
var parent
var radius: float
```

```
func setup(weapon_animation, weapon_damage, weapon_radius, weapon_parent):
    animation = weapon_animation
    damage = weapon_damage
    parent = weapon_parent
    radius = weapon_radius
```

```
func get_collider():
    return $RayCast3D.get_collider()
```

```
func set_sound(audio):
    $AttackSound.stream = audio
```

```
func play_audio():
    $AttackSound.play()
```

flash.gdshader

Found in folder: /shaders

This shader sets the color and transparency of a 3D object. It uses two variables: “color” to set the object’s color, and “alpha” to set its transparency level between 0 (fully transparent) and 1 (fully opaque). The shader applies these values to the object’s material in the fragment function.

```
shader_type spatial;

uniform float alpha: hint_range(0.0, 1.0) = 0.0;
uniform vec3 color: source_color = vec3(1.0);

void fragment() {
    ALBEDO = color;
    ALPHA = alpha;
}
```

grass_blade.gdshader

Found in folder: /shaders

This shader displaces 3D model vertices based on wind and player proximity. It uses texture samplers to control the displacement amount and direction, and uniforms to set colors and player position. The fragment function sets the final color of the model based on the UV coordinates and two uniform colors.

```
shader_type spatial;
render_mode cull_disabled;

uniform vec3 color: source_color;
uniform vec3 shadow_color: source_color;
uniform sampler2D curve_sampler: repeat_disable;
uniform sampler2D wind_sampler: repeat_enable, filter_linear;
global uniform vec3 player_position;

void vertex() {
    // wind displacement
    vec2 world_pos = vec2(NODE_POSITION_WORLD.x, NODE_POSITION_WORLD.z);
    float curve = texture(curve_sampler, vec2(UV.y, 0.0)).x;
    float wind = texture(wind_sampler, world_pos * 0.08 + TIME * 0.05).x - 0.5;
    VERTEX += vec3(wind * curve);

    // player displacement
    vec3 direction_to_player = player_position - NODE_POSITION_WORLD;
    float distance_to_player = length(direction_to_player);
    float falloff = 1.0 - smoothstep(0.0, 1.0, distance_to_player);
    VERTEX += -direction_to_player * falloff * curve * 1.5;
}

void fragment() {
    ALBEDO = mix(color, shadow_color, UV.y);
}
```

```
}
```